

1st April 2026



KINGSTON UNIVERSITY LONDON

Module: Databases & Data Management (CI7230)

Coursework 1: Thames Freight Rail Design Exercise

M.Sc. Data Science

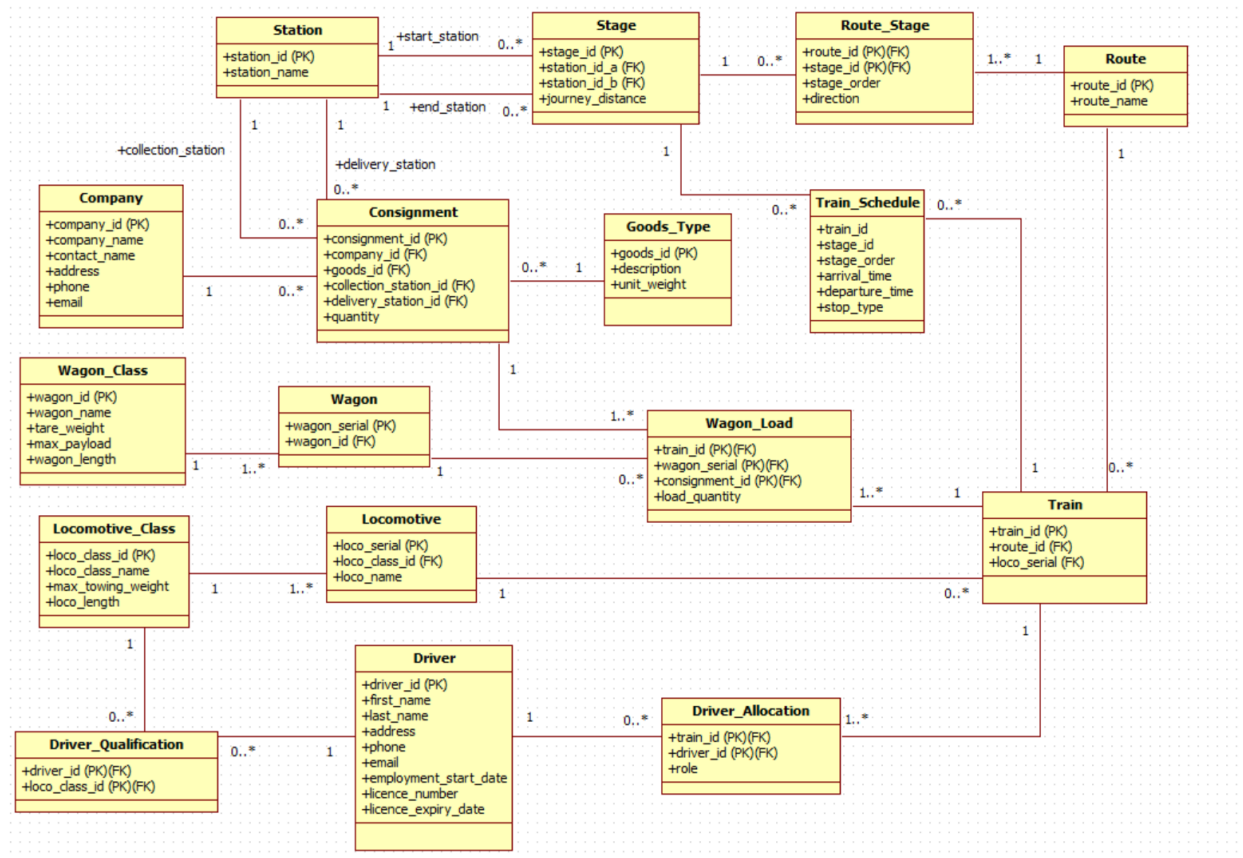
Submitted By

Gooljar Farhaan

KID: 2551126

Chapter 1: The Entity Relationship Diagram with assumptions.

The ER Diagram of the Thames Freight Rail Design is illustrated below:



The ER diagram presents a structured view of a railway freight system by organising its data into interconnected entities that cover network layout, transport operations, and resource management. Key elements such as stations, stages, routes, trains, consignments, wagons, locomotives, and drivers are linked together with clearly defined relationships and cardinalities.

The design incorporates both simple and complex associations. One-to-many relationships capture hierarchical structures like companies having multiple consignments, while many-to-many interactions are handled through associative entities such as `Wagon_Load`, `Train_Schedule` and `Driver_Qualification`. This allows the model to represent scenarios involving shared assets, staged journeys, and distributed cargo.

Data consistency is enforced through the systematic use of primary and foreign keys across all entities. Each table includes attributes that are relevant to its role, contributing to a normalized schema that avoids redundancy while still supporting operational requirements.

Overall, the diagram delivers a coherent and flexible representation of the system, suitable for supporting the complexity of railway freight operations in a database environment.

Constraints and Assumptions

General Rules

- Each entity has a unique primary key.
- All foreign keys must reference existing records, ensuring referential integrity.
- Mandatory relationships must always have a valid reference, while optional relationships may have none.
- Many-to-many relationships are resolved using bridge entities.
- The database is designed in Third Normal Form (3NF) to eliminate redundancy.
- Derived values (e.g., total train length, gross freight weight) are not stored and must be calculated when required.
- All weights are stored in tonnes, lengths in metres, and distances in miles.

Station, Stage, Route and Route Stage

- A stage connects exactly two distinct stations and is considered bidirectional.
- Stage distances are fixed and assumed not to change over time.
- The direction in which a stage is travelled is determined in Route_Stage using the direction attribute. A = station_id_a → station_id_b ; B = station_id_b → station_id_a

Company, Consignment and Goods Type

- Each consignment belongs to exactly one company and represents a single goods type.
- A consignment must have one collection station and one delivery station.
- Quantity represents the number of units, and total weight is derived using *quantity × unit_weight*.
- Detailed consignment information is assumed to be managed by an external system.
- Goods are assigned to wagon classes as follows:
 - Cement, Coal, Steel → Open wagons
 - Cars → Car carriers
 - Perishable goods → Covered wagons
 - Oil → Tank wagons

Train, Wagon, Wagon Load and Consignment

- A train must carry at least one consignment and may carry consignments from multiple companies.
- A consignment must be transported using one or more wagons and may be split across multiple wagons and trains.
- Wagons may be reused across trains at different times but cannot be assigned to multiple trains simultaneously.

Train Schedule

- Schedule entries are recorded only for operational stops (loading/unloading); trains are assumed to pass through other stations without stopping.
- Stage sequences must follow the predefined route order, and duplicate stage entries for a train are not permitted.
- Travel and stop durations are derived from datetime differences.
- Wagons and locomotives may be reused across trains, but scheduling must prevent any time conflicts. Sufficient time gaps must be allowed between train operations.
- The column *stop_type* can only contain exactly one of these three string values: 'START', 'STOP', 'END'.

Locomotive Class and Locomotive

- A locomotive may be used across multiple trains at different times but cannot operate in more than one train simultaneously.
- A locomotive may optionally have a name in addition to its serial number.

Wagon Class and Wagon

- All wagons within the same class share identical characteristics.
- Wagons may be reused across trains at different times but cannot be assigned to multiple trains simultaneously.

Driver, Driver Qualification and Driver Allocation

- Each train must have exactly two drivers, assigned distinct roles (Driver and Co-Driver).
- A driver may be assigned to multiple trains at different times but cannot operate multiple trains simultaneously.
- Licence 10 years validity is enforced using the licence expiry date.

- Each record in Driver_Qualification represents a certification issued to a driver, specifying the locomotive class (rolling stock) they are authorised to operate.
- Each certification is assumed to remain valid as long as the driver's licence is valid.

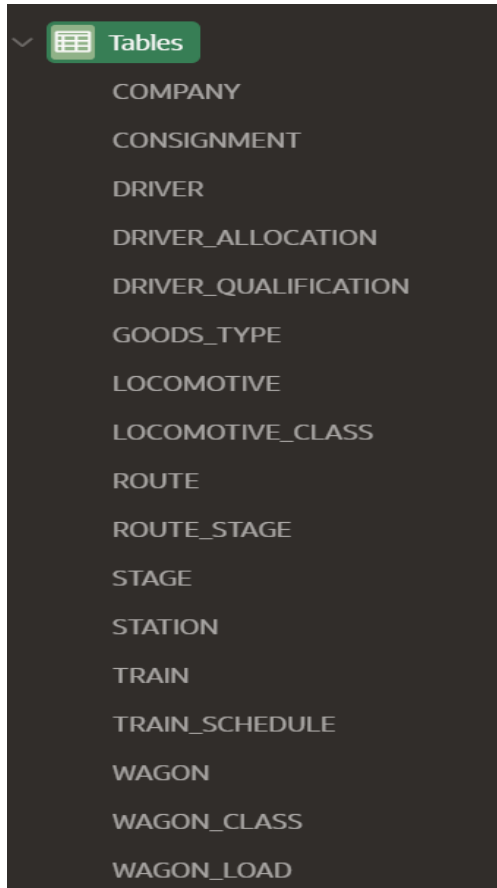
Derived Values

The following values are calculated dynamically and not stored:

- Total train length = locomotive length + sum of wagon lengths
- Gross freight weight = sum of (quantity × unit_weight) across consignments
- The number of Locomotives and Wagons owned, are represented by the number of records in the Locomotive and Wagon datasets respectively.
- All derived values are computed using SQL queries when required

Chapter 2: Implementation. Provide a listing of the SQL table definitions.

Here are all the SQL tables used in the Thames Freight Rail Design:



1) STATION

This table stores the list of railway stations in the network. Each station is uniquely identified by station_id, and the station name is stored for identification purposes. This table serves as a reference point for routes, stages, and consignments.

```
CREATE TABLE Station (
    station_id VARCHAR(10) PRIMARY KEY,
    station_name VARCHAR(50) NOT NULL
);
```

2) STAGE

The Stage table represents a connection between two stations, defining a segment of the railway network. Each stage has a unique identifier and stores the distance between the two stations. It is bidirectional and forms the foundation for route construction.

```
CREATE TABLE Stage (
    stage_id VARCHAR(10) PRIMARY KEY,
    station_id_a VARCHAR(10) NOT NULL,
    station_id_b VARCHAR(10) NOT NULL,
    journey_distance DECIMAL(6,2) NOT NULL,

    FOREIGN KEY (station_id_a) REFERENCES Station(station_id),
    FOREIGN KEY (station_id_b) REFERENCES Station(station_id)
);
```

3) ROUTE

This table defines predefined railway routes, each identified by a unique route_id. A route represents a sequence of stages that a train follows.

```
CREATE TABLE Route (
    route_id VARCHAR(10) PRIMARY KEY,
    route_name VARCHAR(50) NOT NULL
);
```

4) ROUTE_STAGE

The Route_Stage table links routes to stages and defines the order and direction in which stages are travelled. It resolves the many-to-many relationship between routes and stages and ensures proper route sequencing.

```
CREATE TABLE Route_Stage (
    route_id VARCHAR(10) NOT NULL,
    stage_id VARCHAR(10) NOT NULL,
    stage_order INT NOT NULL,
    direction CHAR(1) NOT NULL,

    PRIMARY KEY (route_id, stage_id),
    FOREIGN KEY (route_id) REFERENCES Route(route_id),
    FOREIGN KEY (stage_id) REFERENCES Stage(stage_id)
);
```

5) COMPANY

This table stores information about companies that send consignments.

```
CREATE TABLE Company (  
    company_id VARCHAR(10) PRIMARY KEY,  
    company_name VARCHAR(100) NOT NULL,  
    contact_name VARCHAR(100) NOT NULL,  
    address VARCHAR(200) NOT NULL,  
    phone VARCHAR(20) NOT NULL,  
    email VARCHAR(50) NOT NULL  
);
```

6) GOODS_TYPE

The Goods_Type table categorizes the types of goods being transported. It includes a unit weight attribute, which is used to calculate the total weight of consignments.

```
CREATE TABLE Goods_Type (  
    goods_id VARCHAR(10) PRIMARY KEY,  
    description VARCHAR(50) NOT NULL,  
    unit_weight DECIMAL(10,2) NOT NULL  
);
```

7) LOCOMOTIVE CLASS

This table defines different classes of locomotives, including their physical characteristics and towing capacity. It is used to ensure trains operate within safe limits.

```
CREATE TABLE Locomotive_Class (  
    loco_class_id VARCHAR(10) PRIMARY KEY,  
    loco_class_name VARCHAR(50) NOT NULL,  
    loco_length DECIMAL(5,2) NOT NULL,  
    max_towing_weight DECIMAL(10,2) NOT NULL  
);
```

8) WAGON CLASS

The Wagon_Class table defines different types of wagons, including their dimensions, tare weight, and maximum payload capacity. All wagons of the same class share these properties.

```
CREATE TABLE Wagon_Class (
    wagon_id VARCHAR(10) PRIMARY KEY,
    wagon_name VARCHAR(50) NOT NULL,
    wagon_length DECIMAL(5,2) NOT NULL,
    tare_weight DECIMAL(10,2) NOT NULL,
    max_payload DECIMAL(10,2) NOT NULL
);
```

9) DRIVER

This table stores driver details, including personal information and licence data. Licence expiry is tracked to ensure compliance with operational requirements.

```
CREATE TABLE Driver (
    driver_id VARCHAR(10) PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    date_of_birth DATE NOT NULL,
    address VARCHAR(150) NOT NULL,
    phone VARCHAR(20) NOT NULL,
    email VARCHAR(100) NOT NULL,
    employment_start_date DATE NOT NULL,
    licence_number VARCHAR(20) NOT NULL,
    licence_expiry_date DATE NOT NULL
);
```

10) LOCOMOTIVE

The Locomotive table stores individual locomotives, each linked to a locomotive class. This allows classification and reuse of locomotives across different trains.

```
CREATE TABLE Locomotive (
    loco_serial VARCHAR(10) PRIMARY KEY,
    loco_class_id VARCHAR(10) NOT NULL,
    loco_name VARCHAR(50),
    FOREIGN KEY (loco_class_id) REFERENCES Locomotive_Class(loco_class_id)
);
```

11) WAGON

This table stores individual wagons, each associated with a wagon class. It allows wagons to be tracked and assigned to trains dynamically.

```
CREATE TABLE Wagon (
  wagon_serial VARCHAR(10) PRIMARY KEY,
  wagon_id VARCHAR(10) NOT NULL,
  FOREIGN KEY (wagon_id) REFERENCES Wagon_Class(wagon_id)
);
```

12) CONSIGNMENT

The Consignment table represents goods transported from one station to another. It links companies, goods types, and stations, and stores the quantity of goods being transported.

```
CREATE TABLE Consignment (
  consignment_id VARCHAR(10) PRIMARY KEY,
  company_id VARCHAR(10) NOT NULL,
  goods_id VARCHAR(10) NOT NULL,
  collection_station_id VARCHAR(10) NOT NULL,
  delivery_station_id VARCHAR(10) NOT NULL,
  quantity DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (company_id) REFERENCES Company(company_id),
  FOREIGN KEY (goods_id) REFERENCES Goods_Type(goods_id),
  FOREIGN KEY (collection_station_id) REFERENCES Station(station_id),
  FOREIGN KEY (delivery_station_id) REFERENCES Station(station_id)
);
```

13) TRAIN

This table represents trains operating in the system. Each train is assigned a route and a locomotive, defining its journey and pulling capacity.

```
CREATE TABLE Train (
  train_id VARCHAR(10) PRIMARY KEY,
  route_id VARCHAR(10) NOT NULL,
  loco_serial VARCHAR(10) NOT NULL,
  FOREIGN KEY (route_id) REFERENCES Route(route_id),
  FOREIGN KEY (loco_serial) REFERENCES Locomotive(loco_serial)
);
```

14) TRAIN SCHEDULE

The Train_Schedule table records the operational timetable of trains at specific stages. It includes arrival and departure datetime, as well as the type of stop (start, intermediate stop, or end). This enables tracking of train movement and operational activities.

```
CREATE TABLE Train_Schedule (
    train_id VARCHAR(10) NOT NULL,
    stage_id VARCHAR(10) NOT NULL,
    stage_order INT NOT NULL,
    arrival_time DATE NOT NULL,
    departure_time DATE NOT NULL,
    stop_type VARCHAR(10) NOT NULL,

    PRIMARY KEY (train_id, stage_id),
    FOREIGN KEY (train_id) REFERENCES Train(train_id),
    FOREIGN KEY (stage_id) REFERENCES Stage(stage_id),

    CHECK (stop_type IN ('START','STOP','END'))
);
```

15) DRIVER QUALIFICATION

This table records which locomotive classes each driver is qualified to operate. It ensures that only appropriately trained drivers are assigned to trains.

```
CREATE TABLE Driver_Qualification (
    driver_id VARCHAR(10) NOT NULL,
    loco_class_id VARCHAR(10) NOT NULL,
    PRIMARY KEY (driver_id, loco_class_id),
    FOREIGN KEY (driver_id) REFERENCES Driver(driver_id),
    FOREIGN KEY (loco_class_id) REFERENCES Locomotive_Class(loco_class_id)
);
```

16) DRIVER ALLOCATION

The Driver_Allocation table assigns drivers to trains and defines their roles (Driver or Co-Driver). It supports operational staffing requirements.

```
CREATE TABLE Driver_Allocation (
  train_id VARCHAR(10) NOT NULL,
  driver_id VARCHAR(10) NOT NULL,
  role VARCHAR(20) NOT NULL,
  PRIMARY KEY (train_id, driver_id),
  FOREIGN KEY (train_id) REFERENCES Train(train_id),
  FOREIGN KEY (driver_id) REFERENCES Driver(driver_id)
);
```

17) WAGON LOAD

This table links trains, wagons, and consignments, specifying how goods are distributed across wagons. It enables tracking of freight allocation and supports operational constraints such as weight and capacity.

```
CREATE TABLE Wagon_Load (
  train_id VARCHAR(10) NOT NULL,
  wagon_serial VARCHAR(10) NOT NULL,
  consignment_id VARCHAR(10) NOT NULL,
  load_quantity DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (train_id, wagon_serial, consignment_id),
  FOREIGN KEY (train_id) REFERENCES Train(train_id),
  FOREIGN KEY (wagon_serial) REFERENCES Wagon(wagon_serial),
  FOREIGN KEY (consignment_id) REFERENCES Consignment(consignment_id)
);
```

Chapter 3: A discussion on the data that was entered with screenshots showing that the multiplicity of the relationships are reflected in the data that was entered.

The datasets presented through the following screenshots demonstrates the successful implementation of the railway freight database system, ensuring that relationships, constraints, and operational scenarios are properly represented. All tables have been populated with consistent and meaningful data, validating the correctness of primary and foreign key relationships across the schema. The dataset covers all key operational aspects, including routes, trains, consignments, wagons, and scheduling. It reflects realistic railway logistics scenarios such as multi-stage routes, shared resource utilisation, and consignments distributed across multiple wagons and trains. The screenshots confirm that the database structure supports complex relationships and enforces data integrity, while accurately modelling real-world railway operations.

1- Station Table

STATION		
Columns	Data	Indexes Constraints Grants Statistics Triggers Dependence
+ Insert Row	Columns...	Filter... Count Rows Load Data Download
	STATION_ID	STATION_NAME
	ST1	London
	ST2	Birmingham
	ST3	Manchester
	ST4	Leeds
	ST5	York
	ST6	Newcastle
	ST7	Edinburgh
	ST8	Glasgow
	ST9	Bristol
	ST10	Cardiff
	ST11	Oxford
	ST12	Sheffield
	ST13	Rugby
	ST14	Plymouth

2- Stage Table

- A single station can be connected to multiple stages.
Example: Station 'ST1' appears in the following stages: SG3, SG4, SG14

STAGE

Columns

Data

Indexes

Constraints

Grants

Statistics

Triggers

Dependencies

DDL

Sample Queries

+ Insert Row

Columns...

Filter...

Count Rows

Load Data

Download

Refresh

	STAGE_ID	STATION_ID_A	STATION_ID_B	JOURNEY_DISTANCE
	SG1	ST14	ST9	120
	SG2	ST11	ST9	80
	SG3	ST1	ST11	60
	SG4	ST13	ST1	90
	SG5	ST2	ST13	20
	SG6	ST12	ST2	70
	SG7	ST3	ST12	40
	SG8	ST4	ST3	45
	SG9	ST5	ST4	25
	SG10	ST6	ST5	80
	SG11	ST7	ST6	120
	SG12	ST8	ST7	50
	SG13	ST10	ST9	45
	SG14	ST1	ST2	45

1 cells selected

1 - 14

3- Route Table

ROUTE		
Columns	Data	Indexes Constraints Grants Statistics Trigg
+ Insert Row	Columns...	Filter... Count Rows Load Data
Refresh		
	ROUTE_ID	ROUTE_NAME
	RT1	Glasgow → London
	RT2	Birmingham → Plymouth
	RT3	Bristol → Edinburgh
	RT4	Cardiff → Glasgow
	RT5	Bristol → Manchester
	RT6	Plymouth → Manchester
	RT7	London → Leeds
	RT8	Manchester → Edinburgh
	RT9	Newcastle → Oxford
	RT10	Rugby → Bristol

4- Route Stage Table

- Each route consists of multiple stages.
Example: Route RT1 includes the following SG12 → SG11 → SG10 → ... → SG4 (9 stages)
- A stage can be reused in multiple routes.
Example: Stage SG5 appears in the following Trains: RT1, RT2, etc...

ROUTE_STAGE

Columns

Data

Indexes

Constraints

Grants

Statistics

Triggers

Dependencies

DDL

Sample Queries

+ Insert Row

Columns...

Filter...

Count Rows

Load Data

Download

Refresh

	ROUTE_ID	STAGE_ID	STAGE_ORDER	DIRECTION
	RT1	SG12	1	A
	RT1	SG11	2	A
	RT1	SG10	3	A
	RT1	SG9	4	A
	RT1	SG8	5	A
	RT1	SG7	6	A
	RT1	SG6	7	A
	RT1	SG5	8	A
	RT1	SG4	9	A
	RT2	SG5	1	A
	RT2	SG4	2	A
	RT2	SG3	3	A
	RT2	SG2	4	A
	RT2	SG1	5	B

1 cells selected

1 - 21

5- Train Table

- Multiple trains can operate on the same route.
Example: Route RT1 is used by Train TR1 and Train TR12.
- Each train uses one locomotive, but locomotives can be reused.
Example: Locomotive 8203 is used by Train TR7 and Train TR11.

TRAIN

Columns

Data

Indexes

Constraints

Grants

Statistics

Triggers

Dependencies

DDL

Sample

+ Insert Row

Columns...

Filter...

Count Rows

Load Data

Download

Refresh

	TRAIN_ID	ROUTE_ID	LOCO_SERIAL
	TR1	RT1	7100
	TR2	RT2	8201
	TR3	RT3	9001
	TR4	RT4	8200
	TR6	RT6	7102
	TR7	RT7	8203
	TR9	RT9	8204
	TR10	RT10	7104
	TR11	RT2	8203
	TR12	RT1	7101
	TR13	RT4	8200
	TR14	RT7	8201

6- Company Table

COMPANY

Columns

Data

Indexes

Constraints

Grants

Statistics

Triggers

Dependencies

DDL

Sample Queries

+ Insert Row

Columns...

Filter...

Count Rows

Load Data

Download

Refresh

	COMPANY_ID	COMPANY_NAME	CONTACT_NAME	ADDRESS	PHONE	EMAIL
	CM1	BuildCorp	John Smith	London	0711111111	build@corp.com
	CM2	AutoTrans	Mary Khan	Birmingham	0722222222	auto@trans.com
	CM3	FreshFoods	Ali Noor	Manchester	0733333333	fresh@foods.com
	CM4	OilEnergy	Sara Ali	Glasgow	0744444444	oil@energy.com
	CM5	SteelWorks	Tom Lee	Sheffield	0755555555	steel@works.com

7- Consignment Table

CONSIGNMENT

Columns

Data

Indexes

Constraints

Grants

Statistics

Triggers

Dependencies

DDL

Sample Queries

+ Insert Row

Columns...

Filter...

Count Rows

Load Data

Download

Refresh

	CONSIGNMENT_ID	COMPANY_ID	GOODS_ID	COLLECTION_STATION_IC	DELIVERY_STATION_ID	QUANTITY
	CN1	CM1	GT1	ST8	ST1	500
	CN2	CM2	GT2	ST2	ST14	200
	CN3	CM3	GT3	ST9	ST7	300
	CN4	CM4	GT4	ST10	ST8	450
	CN5	CM5	GT5	ST9	ST3	150
	CN6	CM1	GT6	ST14	ST3	220
	CN7	CM2	GT2	ST1	ST4	180
	CN8	CM3	GT3	ST3	ST7	260
	CN9	CM4	GT4	ST6	ST11	140
	CN10	CM5	GT5	ST13	ST9	120
	CN11	CM1	GT1	ST6	ST1	300
	CN12	CM2	GT2	ST5	ST2	180
	CN13	CM3	GT3	ST4	ST7	200
	CN14	CM4	GT4	ST3	ST8	250

1 cells selected

1 - 16

8- Goods Type Table

GOODS_TYPE

Columns

Data

Indexes

Constraints

Grants

Statistics

Triggers

+ Insert Row

Columns...

Filter...

Count Rows

Load Data

Refresh

	GOODS_ID	DESCRIPTION	UNIT_WEIGHT
	GT1	Cement	1
	GT2	Cars	1.2
	GT3	Perishable goods	.8
	GT4	Mineral oil	1
	GT5	Steel beams	1
	GT6	Coal	1
	GT7	Grain	1
	GT8	Timber	.5

9- Wagon_Class Table

WAGON_CLASS

Columns

Data

Indexes

Constraints

Grants

Statistics

Triggers

Dependencies

DDL

Sample Queries

+ Insert Row

Columns...

Filter...

Count Rows

Load Data

Download

Refresh

	WAGON_ID	WAGON_NAME	WAGON_LENGTH	TARE_WEIGHT	MAX_PAYLOAD
	WC90	Flat wagon	14.6	21	66
	WC91	Open wagon	16.2	33	69
	WC92	Covered wagon	20.6	23.5	66.5
	WC93	Car carrier	24.3	35	15
	WC94	Tank wagon	18.9	27.3	62.7

10- Wagon Table

WAGON

Columns

Data

Indexes

Constraints

Grants

Statistics

Triggers

Dependencies

+ Insert Row

Columns...

Filter...

Count Rows

Load Data

Download

Refresh

	WAGON_SERIAL	WAGON_ID
	90001	WC90
	90002	WC90
	90003	WC90
	90004	WC90
	90005	WC90
	90006	WC90
	90007	WC90
	90008	WC90
	90009	WC90
	90010	WC90
	90011	WC90
	90012	WC90
	90013	WC90

1 cells selected

1 - 135

11- Locomotive_Class Table

LOCOMOTIVE_CLASS

Columns

Data

Indexes

Constraints

Grants

Statistics

Triggers

Dependencies

DDL

+ Insert Row

Columns...

Filter...

Count Rows

Load Data

Download

Refresh

	LOCO_CLASS_ID	LOCO_CLASS_NAME	LOCO_LENGTH	MAX_TOWING_WEIGHT
	LC07	Class 07	16.4	1500
	LC08	Class 08	17.8	1600
	LC09	Class 09	21.4	2000

12- Locomotive Table

1 cells selected

1 - 12

13- Driver Table

DRIVER										
Columns Data Indexes Constraints Grants Statistics Triggers Dependencies DDL Sample Queries										
+ Insert Row Columns... Filter... Count Rows Load Data Download Refresh										
	DRIVER_ID	FIRST_NAME	LAST_NAME	DATE_OF_BIRTH	ADDRESS	PHONE	EMAIL	EMPLOYMENT_START	LICENCE_NUMBER	LICENCE_EXPIRY_DATE...
	DV1	Tom	Hill	5/10/1985	London	0711111111	tom.hill@example...	1/1/2015	L1	1/1/2027
	DV2	Sam	Green	7/15/1987	Manchester	0722222222	sam.green@examp...	1/1/2016	L2	1/1/2028
	DV3	Mark	Stone	3/20/1990	Birmingham	0733333333	mark.stone@exam...	1/1/2017	L3	1/1/2028
	DV4	Luke	West	11/5/1983	Leeds	0744444444	luke.west@examp...	1/1/2014	L4	1/1/2027
	DV5	Ali	Khan	8/25/1992	Bradford	0755555555	ali.khan@example...	1/1/2018	L5	1/1/2029
	DV6	Sara	Noor	12/30/1995	London	0766666666	sara.noor@examp...	1/1/2019	L6	1/1/2028

14- Driver Qualification Table

- A driver can be qualified to drive multiple locomotive classes.
Example: Driver DV1 can drive Locomotive LC07 and Locomotive LC08.
- A locomotive class can be driven by multiple drivers.
Example: Locomotive LC07 can be driven by Driver DV1 and Driver DV2.

DRIVER_QUALIFICATION		
Columns Data Indexes Constraints Grants Statistics Triggers		
+ Insert Row Columns... Filter... Count Rows Load Data Refresh		
	DRIVER_ID	LOCO_CLASS_ID
	DV1	LC07
	DV1	LC08
	DV2	LC07
	DV2	LC08
	DV3	LC08
	DV4	LC08
	DV5	LC09
	DV6	LC09

15- Driver Allocation Table

- Two drivers are always assigned to a train.
Example, Driver DV1 and Driver DV2 drive Train TR1.
- A driver can operate multiple trains.
Example, Driver DV1 drives TrainTR1 and Train TR6.

DRIVER_ALLOCATION

Columns

Data

Indexes

Constraints

Grants

Statistics

Triggers

Dependencies

DDL

+ Insert Row

Columns...

Filter...

Count Rows

Load Data

Download

Refresh

	TRAIN_ID	DRIVER_ID	ROLE
	TR1	DV1	Driver
	TR1	DV2	Co-Driver
	TR2	DV3	Driver
	TR2	DV4	Co-Driver
	TR3	DV5	Driver
	TR3	DV6	Co-Driver
	TR4	DV3	Driver
	TR4	DV4	Co-Driver
	TR6	DV1	Driver
	TR6	DV2	Co-Driver
	TR7	DV3	Driver
	TR7	DV4	Co-Driver
	TR9	DV3	Driver

1 cells selected

1 - 24

16- Wagon Load Table

- A train can use several wagons and a wagon can be used by several trains.
Example: Train TR1 uses multiple wagons: 91013, 93001, 93006, etc... and Wagon 91013 is reused in TR1 and TR12.
- A train can carry multiple consignments.
Example: Train TR1 carry CN11 and CN12.
- A consignment can be transported by several trains and can be split across multiple wagons.
Example: Consignment CN12 is being transported by TR1 and TR12. Consignment CN12 has been split to the following wagons: 93001, 93002, etc...

WAGON_LOAD				
Columns	Data	Indexes	Constraints	Grants
Statistics	Triggers	Dependencies	DDL	
+ Insert Row	Columns...	Filter...	Count Rows	Load Data
Refresh				Download
	TRAIN_ID	WAGON_SERIAL	CONSIGNMENT_ID	LOAD_QUANTITY
	TR1	91013	CN11	24
	TR1	93001	CN12	15
	TR1	93002	CN12	15
	TR1	93003	CN12	15
	TR1	93004	CN12	15
	TR1	93005	CN12	15
	TR1	93006	CN12	15
	TR12	91011	CN1	69
	TR12	91012	CN1	69
	TR12	91013	CN1	62
	TR12	93007	CN12	15
	TR12	93008	CN12	15
	TR12	93009	CN12	15

17- Train Schedule Train

TRAIN_SCHEDULE						
Columns	Data	Indexes	Constraints	Grants	Statistics	Triggers
Dependencies	DDL	Sample Queries				
+ Insert Row	Columns...	Filter...	Count Rows	Load Data	Download	Refresh
	TRAIN_ID	STAGE_ID	STAGE_ORDER	ARRIVAL_TIME	DEPARTURE_TIME	STOP_TYPE
	TR1	SG12	1	3/1/2026	3/1/2026	START
	TR1	SG10	3	3/1/2026	3/1/2026	STOP
	TR1	SG9	4	3/1/2026	3/1/2026	STOP
	TR1	SG6	7	3/1/2026	3/1/2026	STOP
	TR1	SG4	9	3/1/2026	3/1/2026	END
	TR12	SG12	1	3/2/2026	3/2/2026	START
	TR12	SG6	7	3/2/2026	3/2/2026	STOP
	TR12	SG4	9	3/2/2026	3/2/2026	END
	TR2	SG5	1	3/1/2026	3/1/2026	START
	TR2	SG3	3	3/1/2026	3/1/2026	STOP
	TR2	SG1	5	3/1/2026	3/1/2026	END
	TR11	SG5	1	3/2/2026	3/2/2026	START
	TR11	SG1	5	3/2/2026	3/2/2026	END
	TR3	SG2	1	3/1/2026	3/1/2026	START

1 cells selected

1 - 34

Chapter 4: Four queries that demonstrate that your database meets the requirements of the system. Each query should include a short description explaining its purpose and the query's output.

1. Train Compliance Check (Weight & Length Constraints)

The Train Compliance Check query is designed to validate whether each train operates within regulatory and operational constraints. It evaluates two critical conditions: the total gross weight of all wagons (including tare weight and payload) must not exceed the locomotive's maximum towing capacity, and the total length of the train (locomotive plus wagons) must not exceed the 400-metre limit imposed by UK regulations.

The query aggregates wagon weights and lengths and compares them against the locomotive specifications to determine compliance status. The output clearly indicates whether each train is compliant or in violation of these constraints, ensuring that safety and regulatory requirements are consistently enforced within the system.

```

SELECT
  t.train_id as Train_Id,
  SUM(wc.tare_weight + wl.load_quantity) as Total_Wagon_Weight,
  lc.max_towing_weight as Locomotive_Max_Towing_Weight,

  (lc.loco_length + SUM(wc.wagon_length)) as Train_Total_Length,

  400 as Train_Length_Limit,

  CASE WHEN ( SUM(wc.tare_weight + wl.load_quantity) <= lc.max_towing_weight
    AND (lc.loco_length + SUM(wc.wagon_length)) <= 400 ) THEN 'COMPLIANT' ELSE 'NOT COMPLIANT'
  END AS Compliance_Status

FROM Train t
JOIN Locomotive l ON t.loco_serial = l.loco_serial
JOIN Locomotive_Class lc ON l.loco_class_id = lc.loco_class_id
JOIN Wagon_Load wl ON t.train_id = wl.train_id
JOIN Wagon w ON wl.wagon_serial = w.wagon_serial
JOIN Wagon_Class wc ON w.wagon_id = wc.wagon_id

GROUP BY t.train_id, lc.loco_class_name, lc.max_towing_weight, lc.loco_length

ORDER BY t.train_id, lc.loco_class_name

```

TRAIN_ID	TOTAL_WAGON_WEIGHT	LOCOMOTIVE_MAX_TOWING_WEIGHT	TRAIN_TOTAL_LENGTH	TRAIN_LENGTH_LIMIT	COMPLIANCE_STATUS
TR1	1230	1500	324.2	400	COMPLIANT
TR10	186	1500	48.8	400	COMPLIANT
TR11	400	1600	212.2	400	COMPLIANT
TR12	599	1500	210.8	400	COMPLIANT
TR13	308.4	1600	93.4	400	COMPLIANT
TR14	388	1600	212.2	400	COMPLIANT
TR2	639	1600	260.8	400	COMPLIANT
TR3	866.5	2000	248	400	COMPLIANT
TR4	719.2	1600	169	400	COMPLIANT
TR6	601	1500	129.8	400	COMPLIANT
TR7	659	1600	260.8	400	COMPLIANT
TR9	2219	1600	74.5	400	COMPLIANT

2- Driver Qualification Compliance

The Driver Qualification Compliance query verifies that all drivers assigned to trains are qualified to operate the corresponding locomotive class. It checks whether each driver assigned to a train holds a valid qualification for the locomotive class used by that train.

This query ensures that operational safety rules are enforced by preventing unqualified drivers from being assigned to trains. The output highlights any mismatches between driver qualifications and locomotive requirements, demonstrating the system's ability to enforce critical business rules.

SELECT

```

t.train_id as Train_Id,
lc.loco_class_name as Locomotive_Class,
d.driver_id as Driver_Id,
da.role as Driver_Role,
d.first_name || ' ' || d.last_name as Driver_Name,
CASE
    WHEN dq.driver_id IS NOT NULL THEN 'VALID'
    ELSE 'NOT QUALIFIED'
END AS Qualification_Status

```

FROM Train t

JOIN Locomotive l ON t.loco_serial = l.loco_serial

JOIN Locomotive_Class lc ON l.loco_class_id = lc.loco_class_id

JOIN Driver_Allocation da ON t.train_id = da.train_id

JOIN Driver d ON da.driver_id = d.driver_id

LEFT JOIN Driver_Qualification dq ON d.driver_id = dq.driver_id AND lc.loco_class_id = dq.loco_class_id

WHERE d.licence_expiry_date > SYSDATE

ORDER BY t.train_id, d.driver_id

TRAIN_ID	LOCOMOTIVE_CLASS	DRIVER_ID	DRIVER_ROLE	DRIVER_NAME	QUALIFICATION_STATUS
TR1	Class 07	DV1	Driver	Tom Hill	VALID
TR1	Class 07	DV2	Co-Driver	Sam Green	VALID
TR10	Class 07	DV1	Driver	Tom Hill	VALID
TR10	Class 07	DV2	Co-Driver	Sam Green	VALID
TR11	Class 08	DV3	Driver	Mark Stone	VALID
TR11	Class 08	DV4	Co-Driver	Luke West	VALID
TR12	Class 07	DV1	Driver	Tom Hill	VALID
TR12	Class 07	DV2	Co-Driver	Sam Green	VALID
TR13	Class 08	DV3	Driver	Mark Stone	VALID
TR13	Class 08	DV4	Co-Driver	Luke West	VALID

More than 10 rows available. Increase rows selector to view more rows.

3- Wagon Load Status Check

The Wagon Load Status Check query evaluates whether each wagon is operating within its maximum payload capacity, ensuring compliance with safety and operational constraints. Each wagon has a predefined payload limit stored in the Wagon_Class table, and exceeding this limit would represent a critical violation in real-world railway operations.

The query aggregates the total load assigned to each wagon per train using the Wagon_Load table and compares it against the maximum payload defined for its wagon class. Based on this comparison, a status indicator is generated to classify each wagon as either **“OK”** (within capacity) or **“OVERLOADED”** (exceeding capacity).

This query demonstrates the database’s ability to support operational validation by enforcing real-world freight constraints, while also providing a clear and interpretable view of wagon utilisation across trains.

SELECT

wl.train_id as Train_Id,
wl.wagon_serial as Wagon,
wc.wagon_name as Wagon_Name,
wl.load_quantity as Total_Load,
wc.max_payload as Max_Payload,

CASE WHEN wl.load_quantity > wc.max_payload THEN 'OVERLOADED' ELSE 'OK' END AS Load_Status

FROM Wagon_Load wl

JOIN Wagon w ON wl.wagon_serial = w.wagon_serial
JOIN Wagon_Class wc ON w.wagon_id = wc.wagon_id

ORDER BY wl.train_id, wl.wagon_serial

TRAIN_ID	WAGON	WAGON_NAME	TOTAL_LOAD	MAX_PAYLOAD	LOAD_STATUS
TRI	91001	Open wagon	69	69	OK
TRI	91002	Open wagon	69	69	OK
TRI	91003	Open wagon	69	69	OK
TRI	91004	Open wagon	69	69	OK
TRI	91005	Open wagon	24	69	OK
TRI	91009	Open wagon	69	69	OK
TRI	91010	Open wagon	69	69	OK
TRI	91011	Open wagon	69	69	OK
TRI	91012	Open wagon	69	69	OK
TRI	91013	Open wagon	24	69	OK
TRI	93001	Car carrier	15	15	OK
TRI	93002	Car carrier	15	15	OK
TRI	93003	Car carrier	15	15	OK
TRI	93004	Car carrier	15	15	OK

4- Consignment Tracking Query

The Consignment Tracking Query is designed to provide a complete end-to-end view of how consignments are transported across the railway network. It retrieves key operational details including the company responsible for the shipment, the collection and delivery stations, the type and total weight of goods.

In addition, the query identifies all trains involved in transporting the consignment, along with their assigned drivers, locomotives, and the wagons used. It also captures the departure and arrival datetime for each train segment, allowing the tracking of the consignment's movement over time.

This query demonstrates the system's ability to handle complex logistics scenarios where a consignment may be split across multiple wagons and transported by multiple trains, while still maintaining full traceability and data consistency.

SELECT

```
c.consignment_id as Consignment_Id,
comp.company_name as Company,
sc.station_name as Collection_Station,
sd.station_name as Delivery_Station,
gt.description as Goods_Type,
(c.quantity * gt.unit_weight) as Goods_Weight,
t.train_id as Train_Id,
```

```
(SELECT SUM(st2.journey_distance) FROM Route_Stage rs2 JOIN Stage st2 ON rs2.stage_id = st2.stage_id
WHERE rs2.route_id = t.route_id ) AS Journey_Distance,
```

```
MAX(CASE WHEN da.role = 'Driver' THEN d.first_name || ' ' || d.last_name END) AS Driver,
```

MAX(CASE WHEN da.role = 'Co-Driver' THEN d.first_name || ' ' || d.last_name END) AS Co_driver,

t.loco_serial as Locomotive,

LISTAGG(DISTINCT wl.wagon_serial, ', ') as Wagons,

TO_CHAR(MIN(ts.departure_time), 'YYYY-MM-DD HH24:MI') as Departure_time,

TO_CHAR(MAX(ts.arrival_time), 'YYYY-MM-DD HH24:MI') as Arrival_time

FROM Consignment c

JOIN Company comp ON c.company_id = comp.company_id

JOIN Goods_Type gt ON c.goods_id = gt.goods_id

JOIN Station sc ON c.collection_station_id = sc.station_id

JOIN Station sd ON c.delivery_station_id = sd.station_id

JOIN Wagon_Load wl ON c.consignment_id = wl.consignment_id

JOIN Train t ON wl.train_id = t.train_id

JOIN Driver_Allocation da ON t.train_id = da.train_id

JOIN Driver d ON da.driver_id = d.driver_id

JOIN Train_Schedule ts ON t.train_id = ts.train_id

JOIN Route_Stage rs ON t.route_id = rs.route_id

JOIN Stage st ON rs.stage_id = st.stage_id

JOIN Station sa ON st.station_id_a = sa.station_id

JOIN Station sb ON st.station_id_b = sb.station_id

GROUP BY

c.consignment_id, comp.company_name,

sc.station_name, sd.station_name,

gt.description, c.quantity, gt.unit_weight,

t.train_id, t.loco_serial, t.route_id

CONSIGNMENT_ID	COMPANY	COLLECTION_STATION	DELIVERY_STATION	GOODS_TYPE	GOODS_WEIGHT	TRAIN_ID	JOURNEY_DISTANCE	DRIVER	CO_DRIVER	LOCOMOTIVE	WAGONS	DEPARTURE_TIME	ARRIVAL_TIME
CN1	BuildCorp	Glasgow	London	Cement	500	TR1	540	Tom Hill	Sam Green	7100	91001, 91002, 91003, 91004, 91005	2026-03-01 06:30	2026-03-01 20:00
CN1	BuildCorp	Glasgow	London	Cement	500	TR12	540	Tom Hill	Sam Green	7101	91011, 91012, 91013	2026-03-02 11:00	2026-03-02 21:00
CN2	AutoTrans	Birmingham	Plymouth	Cars	240	TR2	370	Mark Stone	Luke West	8201	93001, 93002, 93003, 93004, 93005, 93013, 93014, 93015	2026-03-01 07:30	2026-03-01 14:30
CN2	AutoTrans	Birmingham	Plymouth	Cars	240	TR11	370	Mark Stone	Luke West	8203	93006, 93007, 93008, 93009, 93010, 93011, 93012, 93013	2026-03-02 16:30	2026-03-02 21:00

Chapter 5: Conclusion. A critical evaluation of your final product and a review of the entire exercise.

The final database design successfully models a complex railway freight management system, capturing key operational aspects such as routes, trains, consignments, wagons, drivers, and scheduling. The schema is well-structured and fully normalised, ensuring minimal redundancy and strong data integrity through the appropriate use of primary and foreign keys. Bridge tables such as Wagon_Load and Driver_Allocation effectively resolve many-to-many relationships, enabling the system to represent realistic and flexible operational scenarios.

A key strength of the design is its ability to model complex logistics processes, including multi-stage routes, shared resource utilisation (wagons and locomotives), and consignments distributed across multiple wagons and trains. The introduction of the Train_Schedule table further enhances the system by supporting time-based tracking of train movements, enabling journey analysis, duration calculation, and improved operational visibility. The dataset is comprehensive and consistent, demonstrating correct implementation of relationships and constraints, and reflecting realistic operational rules such as wagon capacity limits, train formation constraints, and staged loading and unloading activities. Furthermore, the SQL queries confirm that the database supports the requirements of the Thames Freight Rail.

However, some limitations exist. Certain constraints such as preventing scheduling conflicts for reused wagons and locomotives, and enforcing strict alignment between schedules and route stages are not fully enforced at the database level and instead rely on careful data design. Additionally, the model incorporates simplified assumptions for travel durations and loading/unloading operations, whereas real-world railway systems involve dynamic conditions such as delays, congestion, and operational variability.

From a personal perspective, this assignment provided valuable experience in designing a complex relational database and translating real-world logistics into a structured data model. It strengthened my understanding of normalisation, relationship modelling, and the importance of clearly defined constraints and assumptions. While the overall structure and dataset design were successful, challenges were encountered in maintaining consistency across interconnected tables and ensuring realistic scheduling without conflicts, requiring iterative refinement. If further improvements were to be made, additional constraint mechanisms such as triggers could be implemented to enforce business rules more strictly, and the scheduling component could be enhanced to better reflect real-world dynamics.

Overall, the assignment was both challenging and highly valuable, demonstrating a strong understanding of database design principles while highlighting opportunities for further enhancement in realism, automation, and constraint enforcement.